Sowing Discord: Exploiting the Lack of Key Commitment in E2EE Group RTC

Hayato Yamashita
The University of Osaka
Suita, Osaka
hayato.yamashita@ist.osaka-u.ac.jp

Hayato Kimura
NICT / The University of Osaka
Suita, Osaka
hayato.kimura@nict.go.jp

Atsushi Tanaka
The University of Osaka
Suita, Osaka
a.tanaka@ist.osaka-u.ac.jp

Takanori Isobe
The University of Osaka
Suita, Osaka
takanori.isobe@ist.osaka-u.ac.jp

***Abstract*—This paper presents a malicious media playback attack against end-to-end encrypted (E2EE) group real-time communication (RTC). The attack makes targeted participants play adversary-chosen audio or video, while non-target participants observe different media-layer outputs from the identical broadcast ciphertext. Crucially, both sides accept the ciphertext as cryptographically authenticated. The attack exploits the lack of key commitment in widely deployed AEAD schemes such as AES-GCM, where a ciphertext may be valid under multiple keys. Unlike prior key-commitment attacks on static data, which allow offline preparation and flexible or multi-format interpretation, our attack targets live group communication. In this setting, the same broadcast ciphertext must be processed immediately as live media under a common codec format, while group key management keeps receivers tightly synchronized. To overcome these obstacles, we formulate a two-phase attack strategy for E2EE group RTC. First, transient TCP delays induce a temporary receiver-dependent key state despite group key synchronization. Second, during this short window, the adversary transmits a codec-aware ambiguous ciphertext within the negotiated media format. Our constructions exploit codec-specific ignorable regions in Opus and H.265, combine them with a cross-key ciphertext layout and AES-GCM collision generation, and avoid brute-force searches, enabling ciphertext generation within live-streaming timing constraints. In particular, for H.265, a single constructed ciphertext can decrypt into two adversary-chosen valid video streams under different receiver keys, subject to the negotiated codec and packetization constraints. We demonstrate the attack against Discord's DAVE protocol through proof-of-concept implementations and feasibility experiments, and show that the audio attack also applies to Linphone. Finally, we identify four conditions required for exploitability: lack of key commitment, asynchronous key updates, tolerant media formats, and absence of authenticated key identification. This analysis guides the design of effective countermeasures.**

1. Introduction

Authenticated Encryption with Associated Data (AEAD) is widely deployed in modern secure communication protocols. In particular, AES-GCM is used in TLS and many application-level encryption schemes because of its efficiency, hardware support, and standardization [1], [2]. It is also used in latency-sensitive communication, including VoIP and WebRTC [3], [4], [5]. However, many standard AEAD schemes, including AES-GCM, lack key commitment [6], [7], [8]. That is, the same ciphertext–tag pair may be valid under multiple keys. An adversary can therefore construct an *ambiguous ciphertext* that authenticates under different keys and decrypts to different plaintexts.

Prior work has shown that this weakness can lead to practical attacks. The Invisible Salamanders attack demonstrated ambiguous AES-GCM ciphertexts against Facebook Messenger attachments [9], and Albertini et al. [10] systematized ambiguous-ciphertext attacks against real systems such as Key Management Services and envelope encryption. These attacks exploit static-data flexibility: ciphertexts can be prepared offline, and key-dependent differences can be hidden using parser-ignored regions or multi-format interpretations such as PDF/JPEG/BMP polyglots.

A natural question is whether the same lack of key commitment can be exploited in end-to-end encrypted (E2EE) group real-time communication (RTC), such as multi-party VoIP and video conferencing. This question is particularly important because E2EE RTC systems now serve as widely used communication infrastructure for privacy-sensitive and high-stakes real-time interactions, where users expect audio and video to be both cryptographically authenticated and consistent across participants [11].

This setting imposes constraints absent from the static-data setting considered in prior attacks. First, group key-management protocols such as the Messaging Layer Security (MLS) protocol [12] enforce epoch-based synchronization, preventing an adversary from freely assigning different keys to receivers. Second, all participants process the same

broadcast media under a common codec format negotiated for the session, precluding multi-format polyglots. Third, the ciphertext must be generated and injected within a short live-communication window, unlike prior static-data attacks that can rely on offline brute-force search or iterative adjustment.

These constraints have led to the perception that key-commitment weaknesses are difficult to exploit in practical E2EE RTC systems.

1.1. Our Contributions

In this paper, we show a malicious media playback attack against E2EE group real-time communication that exploits the lack of key commitment. The attack breaks cross-participant media consistency by causing a target participant to render adversary-chosen audio or video while non-target participants observe different media-layer outputs from the same broadcast ciphertext. In all cases, both sides accept the ciphertext as cryptographically authenticated.

We use Discord's DAVE protocol [13] as our main case study. In DAVE, we demonstrate an Opus audio attack in which the target renders adversary-chosen audio while non-target receivers observe interrupted noise, and an H.265 video attack in which different receivers render distinct adversary-chosen video streams from the same ciphertext. We also analyze Linphone and show that its audio and video settings admit target-specific malicious playback, although not the DAVE-style H.265 construction that yields distinct valid video streams for different receivers. Table 1 summarizes the applicability and protocol features. Our attack requires no malicious server or infrastructure compromise, only a malicious group participant and a network-level adversary capable of delaying the target's inbound TCP flow.

Overall, our results reveal that the lack of key commitment in widely deployed AEAD can undermine cross-participant media consistency in large-scale E2EE real-time communication services such as Discord, making it a practical security issue rather than merely a theoretical concern.

The main contributions of this paper are as follows.

Attack Framework and Enabling Conditions. We formulate a two-phase strategy that turns the lack of key commitment into a practical attack mechanism for live group communication: first creating temporary key-state divergence among receivers, and then transmitting an ambiguous ciphertext during this divergence window.

We identify four design-level conditions that enable the attack: AEAD without key commitment, asynchronous key updates, tolerant media formats, and absence of authenticated key identification. This explains why the attack arises from the interaction between the cryptographic primitive, key-update behavior, and media processing, rather than from a single implementation bug.

Stealthy Key Desynchronization via Network-Level Delay. We show that the required key-state divergence can be induced without compromising the server or modifying protocol messages. The adversary temporarily delays the target's inbound TCP flow carrying key-update control messages, while leaving UDP media traffic uninterrupted. Because the media path continues to operate, the divergence is less visible to users than a full network interruption. The target continues to receive media, but the delayed key update keeps it on an old key while the other receivers transition to a new key. This strategy does not require the network adversary to decrypt TLS, inspect application-layer messages, or identify the Commit message inside the TLS-protected WebSocket stream. Experiments on Discord clients quantify the resulting delay window and show that the key-inconsistent state persists long enough to transmit the ambiguous ciphertext without session teardown.

Fast Ambiguous Ciphertext Generation under a Fixed Codec Format. We construct codec-aware ambiguous ciphertexts under the same negotiated codec format, avoiding the multi-format flexibility used in prior static-data attacks. For Opus, explicit padding hides the corrupted adjustment block, yielding adversary-chosen audio for the target and noise or discarded media for non-target receivers. For H.265, we exploit Filler Data NAL units and DAVE's partial encryption policy to keep the packet acceptable to the WebRTC depacketizer while placing key-dependent corrupted regions in decoder-ignored data. Combined with a cross-keystream layout, start-code avoidance, and GHASH collision generation, this produces *two distinct adversary-chosen video streams from the same ciphertext* in our evaluated DAVE-format WebRTC pipeline. The constructions avoid costly brute-force searches and fit within the measured live-media delay window.

Real-World Feasibility and Experimental Validation. We validate the attack through experiments on real Discord clients and DAVE-format WebRTC media pipelines. These experiments confirm that stealthy key desynchronization can create a delay-induced key divergence while the call remains active, and that ambiguous ciphertexts for Opus and H.265 can be generated and transmitted within this window. These results show that the attack is not merely a cryptographic possibility. The measured timing budget connects the key divergence and media ciphertext components and shows that the attack is feasible under realistic live-communication constraints.

Applicability Across E2EE Calling Systems. We analyze Linphone and show that the attack also applies to both audio and video, causing receiver-dependent media outputs. However, Linphone does not support the DAVE-style H.265 construction that yields two distinct valid video streams from the same ciphertext. This shows that exploitability depends on protocol and implementation details, as further discussed for additional E2EE calling applications.

1.2. Related Work

Discord DAVE. Discord's DAVE protocol uses MLS and WebRTC Encoded Transforms [12], [14] to provide E2EE

TABLE 1. APPLICABILITY OF MALICIOUS MEDIA PLAYBACK ATTACKS ACROSS ANALYZED E2EE APPLICATIONS

Application	Audio Attack	Video Attack	AEAD	Key Upd.	Format	Key Identification	Related Section
Discord	$\checkmark_{\text{single}}$	$\checkmark_{\text{dual}}$	GCM	MLS	Opus, H.265	No KID verification	Sec. 4
Linphone	$\checkmark_{\text{single}}$	$\checkmark_{\text{single}}$	GCM	Original	Opus, H.265	No KID verification	Sec. 5

$\checkmark_{\text{single}}$: the same ciphertext causes the target to render adversary-chosen media, while non-target receivers observe noise, silence, or discarded media; both sides accept it as cryptographically authenticated. $\checkmark_{\text{dual}}$: the same ciphertext causes the target and non-target receivers to render distinct adversary-chosen media outputs; both sides accept it as cryptographically authenticated.

for real-time calls. A 2024 security audit by Trail of Bits [15] identified the lack of key commitment in AES-GCM as a concern, but characterized practical exploitation as unlikely due to the constraints of real-time communication. The audit also noted that excluding certain metadata from the Additional Authenticated Data (AAD) could enable network-level denial-of-service attacks. Our work turns this previously identified but considered hard-to-exploit concern into a concrete malicious media playback attack under a restricted network-delay adversary.

Linphone. Linphone encrypts call media using SRTP with key exchange through SDES, ZRTP, or DTLS-SRTP. Its LIME library, based on the Signal Protocol, targets instant messaging rather than call media [16]. To our knowledge, no prior third-party analysis has examined whether Linphone's group-calling key management and media-encryption metadata binding enable or prevent key-commitment attacks.

2. Preliminaries

2.1. Overview of the DAVE Protocol

The DAVE (Discord Audio/Video Encryption) protocol provides End-to-End Encryption (E2EE) for real-time audio and video calls. Its specifications are published as a whitepaper [13], accompanied by an open-source client implementation, libdave [17], covering media encryption, decryption, and epoch-based key management. The underlying media transport builds upon the WebRTC standard: encrypted audio and video streams are packetized and transported via the Real-time Transport Protocol (RTP), processed through WebRTC's media pipeline including the WebRTC Encoded Transform API [14].

E2EE protocols are generally classified into one-to-one calls and group calls. In one-to-one calls, key agreement is required only between the two parties. However, DAVE targets multi-party group calls in voice channels and group DMs on servers. Therefore, an efficient mechanism for key management and distribution is required.

DAVE adopts AES-GCM for data encryption. The encryption process of AES-GCM is expressed as follows:

$$(C, T) = \text{AES-GCM}(K, N, A, P)$$

Here, K is the encryption key, N is the nonce, A is the Additional Authenticated Data, and P is the plaintext payload. The output is a pair of ciphertext C and authentication tag T. In DAVE, each sender uses its own per-sender symmetric key K, derived from a base secret exported from the shared MLS [12] group state. Receivers reconstruct the same per-sender key from the same MLS export, so all legitimate participants in the group can derive the sender's key at a given epoch.

2.2. Key Update Process in DAVE

DAVE adopts a key management model compliant with the MLS protocol [12], [13]. Members within a group maintain a common state and update the encryption key for each epoch. This section describes the overview of its architecture and the specific key exchange protocol.

2.2.1. Key Management Architecture and Update Model.

1. Media Distribution via Voice Gateway and SFU. The Discord calling system on which the DAVE protocol operates is composed of a two-layer structure: the Voice Gateway, which handles session management, and the SFU (Selective Forwarding Unit), which handles media forwarding. The Voice Gateway is responsible for the delivery of key exchange messages based on the MLS protocol, session management associated with member joining and leaving, and authentication. The SFU handles the routing of actual media data (RTP packets) such as audio and video. Since the SFU does not hold the E2EE decryption keys, it does not concern itself with the content and solely performs the role of forwarding encrypted packets as they are.

In an E2EE group call, the sender encrypts audio data with the group key and transmits the generated pair of ciphertext C and tag T to the SFU. The SFU broadcasts this to all other receivers in the group as shown in Figure 1.

2. Key Updates and Multiple Key Holding Design. In DAVE, key updates are primarily triggered by membership changes, such as when a new member joins or an existing member leaves the group. When a key update occurs, the MLS group state transitions to a new epoch, and a new per-sender media key K_{new} is derived from the newly exported base secret through the DAVE sender key ratchet. However, since real-time calls operate in a distributed manner, not all members can switch keys simultaneously. Therefore, to tolerate network latency and message reordering, decryption clients temporarily hold multiple keys (both new and old). Specifically, even after transitioning to a new epoch, clients retain the old key K_{old} for a maximum of 10 seconds.

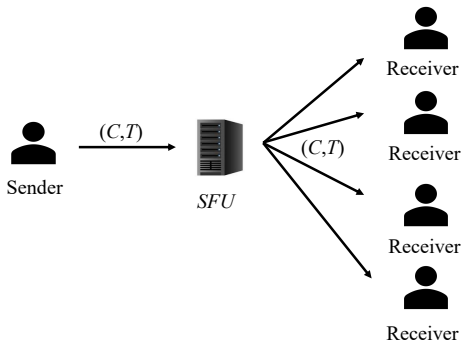


Figure 1. Broadcast by SFU

Since DAVE includes no key-identifying information in the AEAD's Additional Authenticated Data, the receiver cannot determine which key the sender used; instead, it attempts decryption with the latest key K_{new} first and falls back to K_{old} only on failure [15], [17].

2.2.2. Details of the MLS-based Key Exchange Protocol. Let the entities in this protocol be the server S (Voice Gateway, acting as an MLS external sender [12]), an existing group member U_i, a newly joining member U_{new}, and an existing member U_{upd} that acts as a committing member for this epoch transition. Here, we describe the specific technical specifications when U_{new} joins the group and the key is updated among existing members.

1. Key Functions of MLS and DAVE. For clarity of exposition, we denote the relevant MLS library operations and DAVE key management steps by the following abstract pseudo-functions:

- $\mathsf{MLS.KeyPkgGen}() \rightarrow (sk_{new}, pk_{new})$:
  A new member U_{new} generates its private key sk_{new} and public key package pk_{new}.
- $\mathsf{MLS.Commit}(st_u) \rightarrow (st'_u, Commit, Welcome)$:
  The update executor U_{upd} generates a Commit message for state update and a Welcome message for the new member, and transitions its own state to st'_u.
- $\mathsf{MLS.HandleCommit}(st_i, Commit) \rightarrow st'_i$:
  Existing members process the Commit and transition their local group state to st'_i.
- $\mathsf{DAVE.TransitionToKeyRatchet}(kr, t_{exp}) \rightarrow \perp$:
  The decryption client appends the new key ratchet kr to its active list and starts an expiration timer of $t_{exp} = 10\,\mathrm{s}$ for the existing old key, after which the old key is discarded.

2. Protocol Execution Flow. The key update process consists of the following three steps:

1) Preparation for joining and Proposal: U_{new} generates a KeyPackage pk_{new} via MLS.KeyPkgGen and sends it as a join request to the server S. As an MLS external sender, S then constructs an Add proposal containing pk_{new} and broadcasts it to all existing members.
2) Generation and delivery of Commit/Welcome: Upon receiving the broadcast Proposal, U_{upd} (one of the committing members) generates a Commit and a Welcome via MLS.Commit, and sends them to S. S broadcasts the Commit to all existing members U_i and delivers the Welcome directly to U_{new}.
3) State update and key derivation: Upon receiving the message, each member performs the MLS state transition via MLS.HandleCommit and derives new per-sender media keys. To tolerate epoch asynchrony among senders, the client calls DAVE.TransitionToKeyRatchet for each peer's key ratchet, adding the new ratchet while retaining the old one for a brief grace period; existing members U_i thus temporarily hold both the new and old decryption keys (Figure 2).

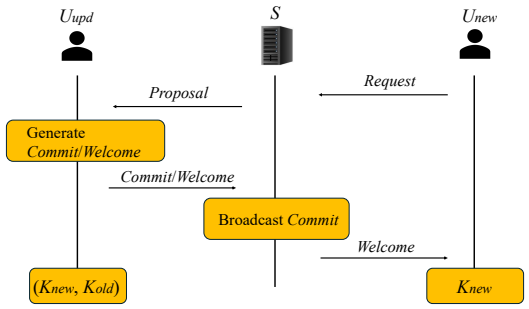


Figure 2. Completion of Key Update (simplified; U_{upd} shown as one of the committing members).

2.3. Key Commitment

Key commitment refers to the property where a pair of ciphertext C and authentication tag T corresponds computationally uniquely to the encryption key K that could have generated it [7], [8]. If this property is lacking, an adversary can successfully authenticate and decrypt a single ciphertext with two different keys [6], [7]. Several security definitions exist for key commitment depending on the scope of what is committed (key, nonce, associated data, etc.). Bellare and Hoang [7] systematized these and showed the following hierarchical relationship:

$$\text{FROB} \Longleftarrow \text{CMT-1} \Longleftarrow \text{CMT-3}$$

Here, the arrows indicate the inclusion relationship of security (definitions become stronger towards the right) [6].

- **CMT-3 (Context Commitment):** The strongest definition, which does not allow collisions between contexts where any of the key K, nonce N, or associated data A differs.

- **CMT-1 (Key Commitment):** A definition that does not allow collisions when the key K is different, while the nonce N and associated data A may differ.
- **FROB (Full Robustness):** A definition that does not allow collisions when the key K is different, and the nonce N and associated data A are fixed (or cannot be controlled by the adversary).

In the attack scenario on real-time communication like DAVE targeted in this paper, the adversary targets the same packet (i.e., identical N, A); thus, the minimum required security requirement corresponds to FROB. Albertini et al. [10] formalized the lack of the most basic FROB security and defined the adversary's advantage as follows:

Definition 1 (Key Commitment Advantage [10]). Let $\Pi = (\mathsf{Enc}, \mathsf{Dec})$ denote an AE scheme. Let $\mathcal{A}$ be an adversary interacting with Π; let $Q_1, Q_2, \ldots$ denote the sequence of queries $\mathcal{A}$ makes to either Enc or Dec, where $Q_i = (K_i, N_i, A_i, M_i, C_i)$ and $\mathsf{Enc}(K_i, N_i, A_i, M_i) = C_i$ or $\mathsf{Dec}(K_i, N_i, A_i, C_i) = M_i$. Then $\mathcal{A}$'s q-KC advantage against Π is the probability that there are two queries Q_i and Q_j where $K_i \neq K_j$, $N_i = N_j$, $C_i = C_j \neq \perp$, $M_i \neq \perp$, $M_j \neq \perp$, and $i, j \leq q$.

Here, $\perp$ represents a decryption error. That is, if the same nonce N and ciphertext C are accepted ($M \neq \perp$) by both sides despite using different keys K_i, K_j, the scheme does not satisfy key commitment. Such a ciphertext that decrypts to valid plaintexts under multiple keys is called an *ambiguous ciphertext* [10].

AES-GCM, adopted in DAVE, fails to satisfy even FROB, the weakest of these notions: collisions exist under different keys with N and A fixed, and hence it does not satisfy key commitment either. This is due to the algebraic structure of the GHASH function used for authentication tag generation. GHASH is designed as a polynomial MAC over GF(2^{128}), which allows an adversary to express the condition for a tag collision as a linear equation. By solving this equation, an adversary can efficiently determine a ciphertext block that satisfies authentication under two different keys simultaneously. The adversary controls all ciphertext blocks except this single adjustment block and does not influence key generation, so the plaintext recovered under either key is adversary-chosen outside that block. We provide the mathematical derivation and the specific linear equation for generating such collisions in Appendix A.

2.4. Key-Commitment Attacks on Multi-Format Static Data

Attacks exploiting the lack of key commitment in AEAD schemes have primarily been studied for static data, where adversaries can prepare ambiguous ciphertexts offline and exploit the structural flexibility of file formats. In particular, prior attacks often rely on multi-format data objects, i.e., byte strings that can be interpreted as different file formats or different semantic contents depending on the decryption key and the resulting plaintext structure. The threat was notably highlighted by Dodis et al. [9] through the Invisible Salamanders attack against Facebook Messenger's attachment encryption. They demonstrated that a single AES-GCM ciphertext can decrypt into two different meaningful images, a JPEG image under one key and a BMP image under another.

Albertini et al. [10] subsequently systematized and generalized this class of vulnerabilities, introducing the notion of ambiguous ciphertexts, in which a single ciphertext decrypts to different meaningful plaintexts under different keys. To induce such semantic divergence, the adversary exploits the structural flexibility of common file formats, such as redundant headers, comment fields, and padding regions. This allows the content revealed under one key to be placed in regions that are ignored or semantically inert under the other key. Using this technique, Albertini et al. demonstrated concrete attacks against Key Management Systems (KMS) and envelope encryption services, where the same ciphertext may be validated as a benign image under one key while being interpreted as a malicious executable under another.

3. Malicious Audio/Video Playback Attack

In this section, we describe a two-phase attack strategy for exploiting the lack of key commitment in E2EE group real-time communication.

The attack first creates a temporary key-state divergence among receivers, and then transmits an ambiguous ciphertext during this divergence window. As a result, a target participant renders adversary-chosen media, while non-target participants observe different media-layer outputs from the same ciphertext. Both sides accept the ciphertext as cryptographically authenticated.

3.1. Adversary Model

We define the capabilities of the adversaries required for the attacks proposed in this paper. Note that the formal definitions of adversary types (Insider, Outsider, and Participant) and security requirement defined by the DAVE protocol [13] are provided in Appendix B.

- A_{net}: An outsider located on the immediate network path of the target user U_{tgt}, such as a Wi-Fi access point, enterprise gateway, or local ISP. A_{net} cannot decrypt TLS, inspect application-layer payloads, modify ciphertexts, or compromise any service infrastructure. Its capability is limited to identifying the target's transport flows from observable metadata and temporarily buffering the inbound TCP flow destined for U_{tgt}.
- U_{mal}: A malicious participant with legitimate privileges in the group call. U_{mal} has already transitioned to the latest key K_{new} and also retains the past key K_{old}. U_{mal} has the capability to generate ambiguous ciphertext.

- U_{trig}: A participant who induces key update events in the protocol by joining or leaving the group call.

We assume that these adversaries fully collude with each other to execute the attack.

3.2. Attack Scenario

We describe the attack scenario independently of a specific implementation. The scenario serves as the basis for the concrete attacks against real-world E2EE communication systems developed in the following sections. The attack consists of two phases: Phase 1 creates a temporary key-state inconsistency among receivers, and Phase 2 transmits an ambiguous ciphertext while that inconsistency persists.

3.2.1. Phase 1: Inducing Key State Inconsistency. First, the adversary establishes a key state inconsistency in which only the target user U_{tgt} in the group remains fixed on the old key K_{old}, while other members U_i transition to the new key K_{new}.

1) Initiation of Key Update: Triggered by U_{trig}'s joining or leaving, the server S initiates the key update process and distributes control messages (such as Commit in MLS) to all members to transition to the new key.
2) Delay of Control Traffic: In coordination with U_{trig}'s action, the network adversary A_{net} temporarily delays the entire control traffic destined for U_{tgt}.
3) Key State Divergence: As a result, U_{tgt} fails to detect the key update and maintains K_{old}, whereas other members, including U_{mal}, process the message normally and transition to K_{new} (Figure 3).

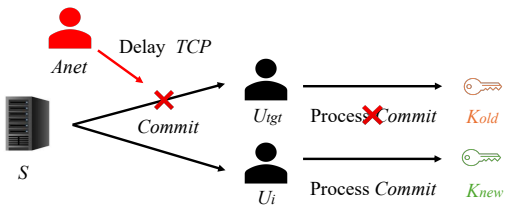


Figure 3. Establishment of a Key State Inconsistency

3.2.2. Phase 2: Generation and Transmission of Ambiguous Ciphertext. Next, U_{mal} exploits the lack of key commitment in AES-GCM to generate and transmit a single ambiguous ciphertext C^*. This ciphertext is constructed to possess the following properties:

1) Cryptographic Integrity: C^* passes valid authentication tag verification regardless of whether it is decrypted with K_{old} or K_{new} (Figure 4).
2) Asymmetric Payload Interpretation:
   - Target U_{tgt}: The decrypted plaintext is interpreted as adversary-chosen audio or video and is played back normally.
   - Other members U_i: The decrypted plaintext is interpreted as audio or video different from that received by U_{tgt}.

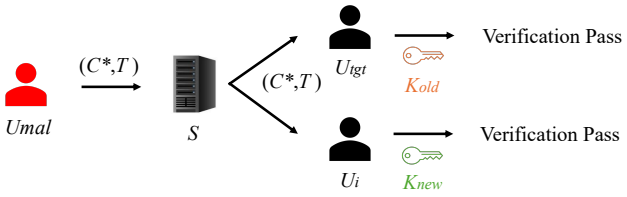


Figure 4. Transmission and Decryption of Ambiguous Ciphertext

In this manner, the adversary causes only the target to play adversary-chosen audio or video, while the other participants receive and play media different from that received by the target from the same ciphertext. The technical challenges required to establish this attack in a real environment are detailed in Section 3.3.

3.3. Challenges in Realizing the Attack

The attack scenario described above requires two components: inducing a temporary key-state divergence among receivers and transmitting an ambiguous ciphertext that is accepted under different keys. Unlike prior key-commitment attacks on static data [9], [10], this must be achieved during live group communication, where media is processed immediately under a negotiated codec.

This setting introduces three challenges.

- **Challenge 1 (Forcing Receiver-Dependent Keys).** Modern E2EE group communication systems use group key agreement protocols such as MLS [12] to keep receivers synchronized. The adversary therefore cannot freely assign different keys to receivers, but must exploit the short transition period during key updates when clients temporarily retain multiple keys. A Trail of Bits audit of DAVE [15] rated this class of attacks as high difficulty.
- **Challenge 2 (Fixed-Codec Constraint).** Static-data attacks [9], [10] can exploit flexible or multi-format file parsing. In E2EE group RTC, however, all receivers process the same media under codecs negotiated for the session, such as Opus or H.264/H.265. Thus, the ambiguous ciphertext must remain valid within the negotiated codec format.
- **Challenge 3 (Short Generation Window).** The key divergence exists only briefly. For example, DAVE retains an old key for at most 10 seconds during epoch transitions [13]. The ambiguous ciphertext must therefore be generated within this live-communication window, ruling out costly search-based constructions.

4. Application to DAVE Protocol

This section instantiates the attack scenario on the actual DAVE protocol and demonstrates how the challenges identified in Section 3.3 are addressed in practice. Specifically, Section 4.1 addresses Challenge 1 by presenting a practical TCP-delay strategy for inducing a temporary key-state inconsistency. Section 4.2 addresses Challenge 2 by constructing single-codec ambiguous ciphertexts for Opus and H.265. Section 4.3.3 addresses Challenge 3 by evaluating whether these ciphertexts can be generated and injected within the available real-time attack window.

4.1. Inducing Key State Inconsistency via Transient TCP Delay

4.1.1. Delay Strategy. To realize the attack scenario defined in Section 3.2, the adversary must induce a temporary key-state inconsistency among receivers. Specifically, the target user U_{tgt} must remain on the old key K_{old}, while the other members transition to the new key K_{new}. In DAVE, participant state-control information, such as camera and mute toggling, and MLS-based key-update messages, namely Commits, are delivered over TCP, while encrypted media streams are transmitted over UDP.

Our strategy exploits this transport separation. The network adversary A_{net} temporarily delays the target's inbound TCP flow while leaving UDP media traffic uninterrupted. Thus, media packets continue to reach U_{tgt}, but the Commit message carrying the key update is delayed. The timing can be coordinated by the colluding participant U_{trig}, who triggers the update by joining or leaving the call. Around this event, A_{net} buffers the inbound TCP flow to U_{tgt} and releases it in order after the chosen delay.

Because the entire TCP flow is delayed, A_{net} need not identify the specific Commit message inside the TLS-protected WebSocket stream. Once the delay is released, the target's TCP/TLS stack processes the buffered data as ordinary delayed traffic.

According to the DAVE specification [13], receiving clients retain the sender key ratchet from the previous epoch for up to 10 s to decrypt in-flight RTP frames. This suggests a 10 s upper bound for recoverability, but the delay tolerated by deployed clients must be measured empirically.

4.1.2. Empirical Validation. To validate the TCP-delay strategy and quantitatively determine the practical attack window, we conducted experiments with multiple Discord clients: a target U_{tgt} on Windows (tested with both the Chrome web client and the desktop application), control observers U_i, and a trigger U_{trig} used solely to join the call and induce MLS epoch transitions.

On the target, we injected artificial inbound-TCP latency with the network simulator clumsy (v0.3) [18] (filter `tcp and inbound`, Lag mode), sweeping the delay X from a few hundred milliseconds up to 15 s, while monitoring internal MLS state transitions via DevTools [19].

Screen recordings of these experiments, the full setup, and the DevTools logs are provided in our artifact[1]; it confirms the per-delay recovery behavior and the epoch-transition timing reported here, and following its documented procedure reproduces these results.

To isolate the behavior of key-exchange messages, we fixed all participant states in advance and ensured no camera or microphone toggling occurred during the delay injection, since such control messages also travel over TCP.

4.1.3. Delay Tolerance and Attack Window. By varying the delay duration X, we observed three behavioral regimes.

- $X \leq 4$ s: The target client remained stranded on K_{old} for X s, temporarily losing media synchronization with other members who had already transitioned to K_{new}. However, once the delay elapsed, the client automatically processed the Commit message and naturally resumed the call without disconnections (demonstrated in `TCP_Delay_4s_Recovery.mp4`[1]).
- $5 \text{ s} \leq X \leq 10$ s: In addition to this temporary media desynchronization, a platform-specific limitation emerged. For the web version, the target failed to process the delayed Commit message, ultimately resulting in a session disconnection and an automatic reconnection attempt (demonstrated in `TCP_Delay_5s_Disconnect.mp4`[1]). In contrast, the desktop version successfully processed the Commit message and restored the session seamlessly.
- $X \geq 10$ s: The desktop version's TCP stack could handle delays of up to 15 s. However, exceeding the 10 s threshold simply reflected DAVE's specified grace period for retaining the old key. Because the other group members discarded the old key after 10 s while the delayed target remained on it, the other members lost the ability to decrypt the target's media streams.

These results show that the required key-state inconsistency can be maintained for up to 4 s across the tested clients without session teardown. As shown in Figure 5, this creates an attack window in which the target remains on K_{old} while the ambiguous ciphertext C^* is transmitted.

4.2. Constructing Fast Ambiguous Ciphertexts under a Fixed Codec

DAVE uses Opus as its audio codec and supports video codecs such as H.264/H.265, VP8/VP9, and AV1 [13].

Prior key-commitment attacks can hide key-dependent corrupted blocks using flexible or multi-format static data, such as JPEG/BMP polyglots or parser-ignored regions [9], [10]. This flexibility is unavailable in live RTC: all receivers process the same ciphertext immediately under a single negotiated codec. Therefore, key-dependent corrupted regions

1. https://doi.org/10.5281/zenodo.22764842

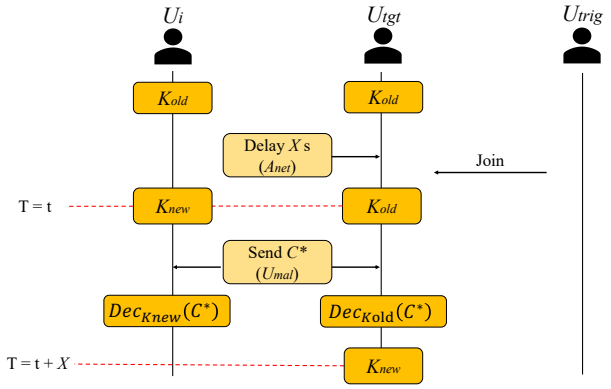


Figure 5. Overview of inducing key state inconsistency and the attack timeline. The network adversary A_{net} temporarily delays the entire TCP traffic destined for U_{tgt} for X seconds. Within this attack window, U_{tgt} remains stranded on K_{old} while other members transition to K_{new}, allowing U_{mal} to transmit the ambiguous ciphertext C^*.

must be confined to codec elements that are ignored, safely skipped, or perceived as routine noise.

We instantiate this idea for Opus and H.265. For Opus, explicit padding hides the corrupted adjustment block under the target's key, yielding adversary-chosen audio for the target and interrupted noise for non-target receivers. For H.265, given two adversary-chosen streams compatible with the negotiated codec and packetization constraints, our construction produces a single ciphertext that decrypts and decodes into one stream under one receiver key and into the other stream under another receiver key.

4.2.1. Attack on Opus Audio Codec.

Opus Padding Specification. An Opus packet begins with a 1-byte TOC (Table of Contents) header specifying the encoding configuration. When the frame-packing code is set to Code 3 (least significant bits `11`), a subsequent frame count byte specifies whether padding is enabled. If enabled (`p=1`), padding bytes are appended to the end of the packet, their size explicitly declared by a padding length indicator (see Figure 6 for the exact packet structure). According to the standard, the Opus decoder safely ignores and discards this entire padding region before decoding the actual audio payload [20].

Attack Strategy and Algorithm. We exploit Opus padding to conceal the AES-GCM tag collision. As shown in Algorithm 1, we construct a Code 3 packet and reserve the final 16-byte padding block as a dummy block. After solving the GHASH collision, the resulting adjustment block replaces this dummy block. Under the target's key, the corrupted block lies inside the declared padding region and is discarded before audio decoding. The payload is encrypted with the keystream derived from K_{old} (Step 3), so the target recovers the adversary-chosen audio, and the tag is finally computed under K_{old} and truncated to the DAVE format (Step 5).

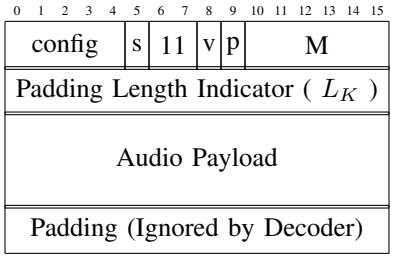


Figure 6. Structure of a Code 3 Opus packet with padding enabled. The TOC byte indicates Code 3 (the least significant 2 bits are `11`), and the frame count byte sets `v=0` (CBR), `p=1` (padding enabled), and `M=1` (single frame). The trailing padding region is safely ignored by the decoder.

The authentication tag is truncated to 8 bytes to comply with the DAVE protocol, although the attack functions flawlessly with standard 16-byte tags as well. For complete mathematical derivations and efficient implementation details, refer to our Proof of Concept (PoC) repository[1].

Algorithm 1 Generation of Ambiguous Ciphertext for Opus

Require: Original Payload M, Keys K_{old}, K_{new}, Base Nonce IV_{base}, SeqNum seq, AAD
Ensure: Malicious Packet ($seq \parallel T_{trunc} \parallel C^*$)
Step 1: TOC and Header Modification
$\text{TOC}_{new} \leftarrow \text{TOC}_{old} \vee$ 0x03 {Force Code 3}
$fc_byte \leftarrow$ 0x41 {Set CBR, $p = 1$, 1 frame}
Step 2: Padding Calculation and Plaintext Construction
$L_{head} \leftarrow 3$ {Header length (TOC, fc_byte, L_K)}
$L_{pad} \leftarrow 16 \times \lceil (|M| + L_{head})/16 \rceil - (|M| + L_{head}) + 16$
$L_K \leftarrow \text{EncodePaddingLength}(L_{pad})$
$P_{tgt} \leftarrow \text{TOC}_{new} \parallel fc_byte \parallel L_K \parallel M$
Step 3: Keystream Encryption and Padding Reservation
$IV \leftarrow IV_{base} + seq$
$KS_{old} \leftarrow \text{AES-CTR-KS}(K_{old}, IV, |P_{tgt}|)$
$C_{base} \leftarrow P_{tgt} \oplus KS_{old}$
$C_{tmp} \leftarrow C_{base} \parallel 0^{L_{pad}}$
Step 4: GHASH Collision and Tag Adjustment
Identify adjustment block index j corresponding to the trailing 16-byte dummy block
$C^*[j] \leftarrow \text{SolveGHASH}(C_{tmp}, AAD, K_{old}, K_{new})$
$C^* \leftarrow C_{base} \parallel 0^{L_{pad}-16} \parallel C^*[j]$
Step 5: Tag Truncation and Packet Assembly
$J_0 \leftarrow IV \parallel$ `0x00000001`
$T \leftarrow \text{GHASH}(C^*, AAD, K_{old}) \oplus \text{AES}(K_{old}, J_0)$
$T_{trunc} \leftarrow T[0:8]$ {Truncate tag to 8 bytes}
return $seq \parallel T_{trunc} \parallel C^*$

Playback Results. We verified this construction using our PoC[1], which sequentially transmits the generated ambiguous ciphertexts. As illustrated in Figure 7, cryptographic verification succeeds under both keys. Under K_{old}, the target receiver plays the clear, adversary-chosen audio because the corrupted adjustment block is confined to the Opus padding region and discarded before decoding. Under K_{new},

the plaintext is pseudo-randomized; the randomized headers either fail Opus structural checks or feed pseudo-random data to the SILK/CELT layers, resulting in interrupted noise in our PoC. Thus, the same ciphertext authenticates under both keys while producing receiver-dependent audio-layer behavior.

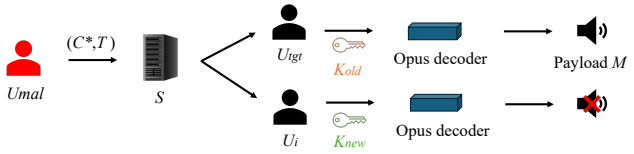


Figure 7. Overview of the playback results. The malicious participant (U_{mal}) transmits an ambiguous ciphertext stream. The target decrypts it using K_{old} and plays the clear, adversary-chosen audio. In contrast, other members decrypt it with K_{new} and hear an interrupted noise due to decoding errors. Both cases successfully pass the cryptographic tag verification.

4.2.2. Attack on H.265 Video Codec. Unlike the Opus attack, which produces interrupted noise under the unintended key, H.265 enables a stronger construction: the same ciphertext can decrypt and decode into two distinct adversary-chosen video streams under different receiver keys.

The same idea also applies to H.264 [21], due to its similar NAL-unit syntax and Filler Data semantics. We focus on H.265 because its higher compression efficiency yields smaller plaintexts and faster ciphertext generation within the real-time delay window.

H.265 Bitstream and Decoder Resilience. According to Annex B of the ITU-T H.265 specification [22], an H.265 byte stream S is composed of a continuous sequence of Network Abstraction Layer (NAL) units, each prefixed by the 3-byte start code (`0x000001`).

$$S = SC \parallel NAL_1 \parallel SC \parallel NAL_2 \parallel \dots$$

Real-world decoders exhibit strong error resilience by safely skipping unrecognized byte sequences until the next valid start code is detected [23], [24].

Naive Polyglot Strategy. Exploiting this error resilience, an adversary can construct a naive polyglot by interleaving chunks from two distinct video streams (Video A and Video B), denoted as $A_{chunk,i}$ and $B_{chunk,i}$, ensuring each chunk begins with a start code.

When the resulting ciphertext is decrypted with the target's key (K_{old}), the chunks corresponding to Video A emerge intact, while the chunks corresponding to Video B turn into pseudo-random corrupted sequences (denoted as R). Conversely, decrypting with the other key (K_{new}) symmetrically yields intact Video B chunks and corrupted Video A chunks. Figure 8 illustrates this symmetric structure.

We verified with our PoC[1] that this naive polyglot works in local decoders such as GStreamer when the pseudo-random regions are kept free of accidental start codes (Algorithm 2).

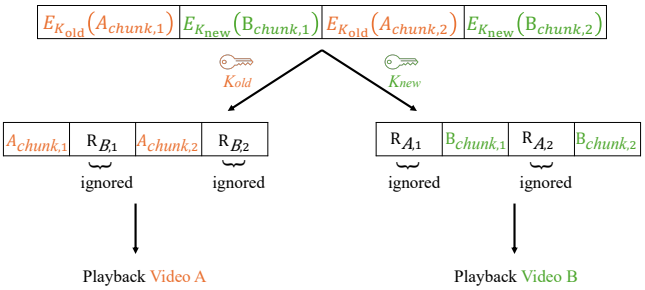


Figure 8. Asymmetric decryption and decoding of the naive polyglot ciphertext. When decrypted under K_{old}, chunks of Video A are decoded normally while corrupted Video B chunks are safely ignored by the decoder. Under K_{new}, the roles are perfectly reversed.

WebRTC Limitations and ENV-Based Polyglot. Although the naive polyglot works in local decoders, transmitting H.265 over RTP in WebRTC imposes an additional constraint before decoding: the RTP depacketizer inspects the 2-byte NAL-unit header at the beginning of each payload [25]. In the naive construction, this header may become pseudo-random under the unintended key, causing the WebRTC stack to discard the packet before it reaches the video decoder.

To overcome this limitation, we exploit DAVE's partial encryption policy [13], [17]. DAVE leaves the NAL-unit headers of non-VCL NAL units, i.e., `nal_unit_type` ≥ 32, unencrypted while authenticating them as AAD. We therefore construct an unencrypted envelope (ENV) using an H.265 Filler Data NAL unit (FD_NUT, `nal_unit_type` = 38). The ENV exposes a valid NAL-unit header to the RTP depacketizer, while the encrypted payload that follows contains the key-dependent regions of the polyglot. Figure 9 shows the resulting packet layout, the range covered by encryption, and the media recovered under each receiver key.

Filler Data NAL units are intended for bitrate adjustment using dummy padding bytes. We exploit the fact that the filler payload is semantically irrelevant to playback: in our evaluated decoders, the payload is skipped until the next valid start code is encountered. This allows us to place the pseudo-random corrupted regions inside the Filler Data body, where they are ignored by the decoder rather than interpreted as video content.

During encryption, we also apply a start-code avoidance loop to ensure that the corrupted regions contain no accidental start code (`0x000001`). This prevents the decoder from prematurely recognizing a false NAL-unit boundary inside the pseudo-random data. Together, DAVE's authenticated but unencrypted NAL-unit metadata, the decoder-ignored Filler Data body, and start-code avoidance allow the ENV-based polyglot to pass RTP depacketization and decode correctly in the WebRTC media pipeline.

Concrete Algorithm. We now give the concrete algorithm for the ENV-based polyglot construction. The construction hides the corrupted regions required by the AES-GCM tag

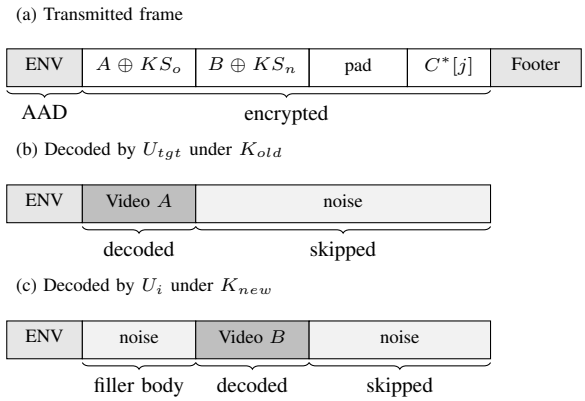


Figure 9. Layout of an ENV-based polyglot frame. KS_o and KS_n denote the keystreams under K_{old} and K_{new}, and *pad* denotes the 16-byte alignment padding. The ENV, i.e., the start code (`0x000001`) and the Filler Data NAL header (`nal_unit_type` = 38), is unencrypted and authenticated as AAD, so the RTP depacketizer always sees a valid NAL-unit header. Under K_{old}, Video A carries its own start code, so the filler body is empty and A is decoded directly; under K_{new}, the leading region becomes the filler body and decoding resumes at Video B.

collision inside the body of a Filler Data NAL unit, which is skipped by the decoder after its header is parsed. As detailed in Algorithm 2, we declare the ENV header as the AAD region and prepare the plaintext as a 16-byte-aligned concatenation of one chunk from Video A and one chunk from Video B (Steps 1–2).

The ciphertext is then constructed using a cross-keystream layout: the Video A region is encrypted with the keystream derived from K_{old}, while the Video B region is encrypted with the keystream derived from K_{new}. As a result, decryption under K_{old} recovers Video A, whereas decryption under K_{new} recovers Video B. For each candidate nonce, we solve the GHASH collision and check that no accidental start code appears in the final ciphertext C^*, including the adjustment block, or in either decrypted corrupted region (Step 3).

After a valid nonce is found, the dummy block is replaced by the GHASH adjustment block, making the tag verify under both keys. Finally, we truncate the tag to 8 bytes and assemble the packet with the DAVE footer (Step 4–5). Complete implementation details are provided in our PoC[1].

Playback Results. We evaluated the ENV-based polyglot in our PoC[1] built on the same media stack that DAVE uses: a WebRTC `RTCPeerConnection` with H.265 enforced via `setCodecPreferences`, and the WebRTC Encoded Transform API (`createEncodedStreams`) carrying DAVE-format frames (i.e., partial encryption with the unencrypted ranges authenticated as AAD and an 8-byte truncated GCM tag). To emulate the SFU broadcast, the identical polyglot was injected toward two receivers configured with K_{old} and K_{new} over separate sender–receiver connections. Both clients successfully decrypted the stream via their receive-side transforms and rendered it through the browser's native RTP depacketizer and H.265 decoder.

Algorithm 2 Generation of an ENV-Based Polyglot Ciphertext

Require: Video A chunk A, Video B chunk B, Keys K_{old}, K_{new}, Base Nonce IV_{base}, Frame ID id
Ensure: Malicious Video Packet ($ENV \parallel C^* \parallel Footer$)
Step 1: Envelope and AAD Construction
$ENV \leftarrow$ `0x000001` $\parallel$ `0x4C01` {Start code + Filler-Data NAL header (`nal_unit_type`=38); unencrypted, authenticated as AAD}
$AAD \leftarrow ENV$
Step 2: Plaintext Alignment
$P_{base} \leftarrow$ pad16($A \parallel B$) {Zero-pad to a 16-byte boundary}
Step 3: Collision-Coupled Start-Code Avoidance Loop
for $attempt = 0$ **to** $maxRetry - 1$ **do**
  $IV \leftarrow (IV_{base} \oplus id) + attempt$
  $KS_{old} \leftarrow$ AES-CTR-KS($K_{old}, IV, |P_{base}| + 16$)
  $KS_{new} \leftarrow$ AES-CTR-KS($K_{new}, IV, |P_{base}| + 16$)
  $C_{tmp} \leftarrow (A \oplus KS_{old}[0 : |A|]) \parallel (B \oplus KS_{new}[|A| : |A| + |B|]) \parallel KS_{new}[|A| + |B| : |P_{base}|] \parallel 0^{16}$
  $j \leftarrow$ index of the trailing 16-byte block of C_{tmp} {the reserved 0^{16} dummy block}
  $C^*[j] \leftarrow$ SolveGHASH($C_{tmp}, AAD, K_{old}, K_{new}$)
  $C^* \leftarrow C_{tmp}$ with block j replaced by $C^*[j]$
  $N_{old} \leftarrow C^* \oplus KS_{old}$ {recovers A in $N_{old}[0 : |A|]$; noise elsewhere}
  $N_{new} \leftarrow C^* \oplus KS_{new}$ {recovers B in $N_{new}[|A| : |A| + |B|]$; noise elsewhere}
  if `0x000001` $\notin C^*$ **and** $\notin N_{old}[|A| :]$ **and** $\notin N_{new}[0 : |A|]$ **and** $\notin N_{new}[|P_{base}| :]$ **then**
    break {No start code in the ciphertext, the adjustment block, or either noise region}
  end if
end for
Step 4: Tag Finalization
$J_0 \leftarrow IV \parallel$ `0x00000001`
$T \leftarrow$ GHASH(C^*, AAD, K_{old}) $\oplus$ AES(K_{old}, J_0)
$T_{trunc} \leftarrow T[0 : 8]$ {Truncate tag to 8 bytes}
Step 5: Packet Assembly
$Footer \leftarrow$ Build-DAVE-Footer($T_{trunc}, IV, \dots$)
return $ENV \parallel C^* \parallel Footer$

As illustrated in Figure 10 and demonstrated in `Playback.mp4` within our PoC[1], both receivers validated the 8-byte tags; the receiver holding K_{old} seamlessly played Video A, while the receiver holding K_{new} cleanly played Video B, with neither client experiencing visual noise or decoding crashes. Because the key-dependent corrupted regions are encapsulated within the encrypted Filler Data payload and sheltered behind the valid, unencrypted ENV header, the frame successfully passed through the live WebRTC RTP depacketizer and, once decrypted in the receive-side transform, decoded cleanly, demonstrating its viability beyond merely a standalone local decoder.

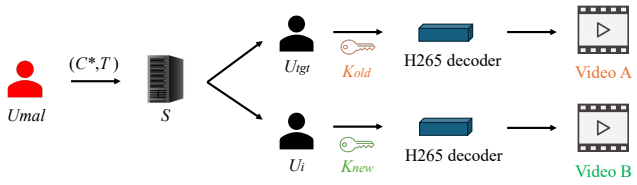


Figure 10. Overview of the H.265 video polyglot playback results. The malicious participant (U_{mal}) transmits an ambiguous ciphertext stream. The target decrypts it using K_{old} and seamlessly plays Video A. In contrast, other members decrypt it with K_{new} and cleanly play Video B. Both cases successfully pass the cryptographic tag verification and experience no visual noise.

4.2.3. Computational Efficiency of Ciphertext Generation. To enable on-the-fly execution, our approach eliminates computationally expensive brute-force searches by exploiting codec-specific structural properties. For the Opus codec, confining the single corrupted block to the padding region removes the need for any brute-force search, so a single GHASH collision suffices. For the H.265 codec, the start-code avoidance loop relies on the fact that the probability of a 3-byte pseudo-random sequence accidentally matching the start code (`0x000001`) is only 2^{-24}. Thus, the nonce tweaking process typically resolves in a few iterations.

Our PoC experiments confirm this efficiency. The two core operations are GHASH collision resolution and the start-code avoidance loop. Both scale linearly with the input file size N ($\mathcal{O}(N)$), so the ciphertext generation time is highly predictable. For the Opus audio attack, generating the ambiguous ciphertext took 0.54 s for a 2 s clip (258 KB), and up to 2.81 s for a 14 s clip (1.4 MB). For the H.265 video attack, generating the core cryptographic collision for a polyglot weaving two distinct video streams required only 0.40 s for 5 s streams (1.0 MB in total), and 0.82 s for 10 s streams (2.1 MB in total).

4.3. Real-World Feasibility and Impact

In this section, we now evaluate whether the two attack components established above compose under realistic conditions, focusing on the required adversary, detectability, timing budget, and impact on group calls.

4.3.1. Realistic Adversary Instantiation. A natural concern is whether the three roles in our attack model (U_{mal}, A_{net}, and U_{trig}) require complex coordination among independent entities. In practice, these logical roles can be easily consolidated into a single human adversary. The adversary can use two separate accounts to simultaneously act as both U_{mal} and U_{trig}. If this adversary also has access to the target's local network, they can concurrently assume the role of A_{net}. This consolidation eliminates the need for strict network-based synchronization, allowing the adversary to initiate the TCP delay locally and immediately trigger the key update with their second account.

This single-adversary execution is highly practical because the required capability of A_{net} is strictly limited. A_{net} does not need to decrypt TLS, inspect application-layer messages, inject protocol messages, modify ciphertexts, or compromise the service infrastructure. It only needs to temporarily delay the inbound TCP flow destined for U_{tgt}, based on observable transport metadata such as IP addresses, ports, and connection timing.

The required capability is weaker than a classical Dolev-Yao adversary [26]: the adversary does not need to read, inject, or alter protocol messages. It also aligns with standard network adversarial models used in formal analyses of modern E2EE messaging protocols [27], [28], following the universally composable adversary formalization [29]. Most importantly, this capability falls within DAVE's own threat model, which considers outsiders capable of intercepting and modifying network traffic [13]. In practice, delay-only interference is within the capabilities of entities with local network control, such as Wi-Fi access points, enterprise middleboxes, or local ISPs that actively shape or buffer customer traffic [30], [31], [32].

4.3.2. Multi-Layered Stealthiness. The attack is difficult to detect across protocol, application, and perceptual layers. At the protocol level, the delay causes no session reset within the measured safe window; once the delay is released, the target's TCP/TLS stack processes the buffered Commit message as ordinary delayed traffic and restores epoch synchronization. At the application level, because A_{net} delays only TCP traffic, encrypted UDP media streams continue to reach the target. As shown in Section 4.1 and our comparative demonstration (`TCP_Delay_4s_Recovery.mp4` vs. `TCP_UDP_Delay_Warning.mp4`)[1], this TCP-only delay leaves the media path uninterrupted and does not trigger visible connection-quality warnings in our experiments.

At the perceptual level, the target renders adversary-chosen media, while non-target receivers observe outputs consistent with the negotiated codec. In the Opus attack, they observe interrupted noise, which can be mistaken for packet loss within a short attack window. In the H.265 attack, they render a different valid video stream from the same ciphertext. In presentation-style scenarios where only the malicious participant transmits media, the target need not observe frozen video or decryption failures from other honest participants while remaining on K_{old}.

Together, these properties make the attack hard to distinguish from ordinary network or media artifacts. The target and non-target receivers observe different media-layer outputs from the same ciphertext, while both sides accept it as cryptographically authenticated.

4.3.3. End-to-End Attack Timeline and Compositional Feasibility. We assess end-to-end feasibility through the timing budget of Table 2. The delay need not be tightly synchronized: A_{net} can begin buffering U_{tgt}'s inbound TCP shortly before U_{trig} triggers the epoch transition. Since U_{mal} starts generation only after observing that transition, C^* is emitted after the honest members activate K_{new} and while U_{tgt} remains on K_{old}. Thus, both receiver key conditions hold when C^* is processed.

TABLE 2. TIMING BUDGET FOR THE COMPOSED ATTACK.

Step	Actor	Time	Src.
Trigger join/leave	U_{trig}	~ 0 s	–
Begin TCP buffering	A_{net}	~ 0 s	–
Detect epoch transition	U_{mal}	≤ 0.31 s	Artifact[1]
Gen. Opus ciphertext (2 s clip)	U_{mal}	0.54 s	Sec. 4.2.3
Gen. H.265 ciphertext (5 s streams)	U_{mal}	0.40 s	Sec. 4.2.3
Transmit to SFU	U_{mal}	< 0.1 s	[33]
Total elapsed (Opus)	–	$\sim \mathbf{0.95}$ s	–
Total elapsed (H.265)	–	$\sim \mathbf{0.81}$ s	–
Available window	–	$\mathbf{4.0}$ s	Sec. 4.1
Margin (Opus)	–	$\mathbf{3.05}$ s	–
Margin (H.265)	–	$\mathbf{3.19}$ s	–

Ciphertext generation dominates the budget at 0.54 s for Opus and 0.40 s for H.265 (Section 4.2.3); the remaining steps are minor. From our demonstration[1] over 39 epoch transitions, epoch activation completes within 0.31 s of observing the update. This is conservative, since a passive U_{mal} incurs only the shorter commit to activation portion, and SFU forwarding adds only milliseconds [33]. The resulting totals in Table 2 leave over 3 s of margin within the 4.0 s window of Section 4.1.

A fully integrated experiment would require Discord's production Voice Gateway and SFU behavior. Since these server-side components are proprietary, we do not reproduce a full independent DAVE deployment, and we refrain from attacking Discord's production service for ethical reasons. Instead, we validate the two attack-relevant components separately: Phase 1 on real Discord clients (Section 4.1) and Phase 2 on a DAVE-format WebRTC media path (Section 4.2.2).

These separated experiments compose through a single attack interface. Phase 1 establishes the required receiver key state, where the target remains on K_{old} while the other receivers have transitioned to K_{new}, together with a measured key divergence window. Phase 2 requires exactly this key state, and uses this interface to construct and transmit an ambiguous ciphertext accepted under the two receiver keys.

Reproducing Discord's proprietary server-side implementation is not needed to validate this interface. The properties exercised by the attack, namely bounded old key retention, trial decryption without authenticated key identification, and DAVE-format partial encryption, are client side behaviors specified in the DAVE whitepaper [13] and implemented in libdave [17]. The relevant server-side role is to deliver key update control traffic and forward encrypted media packets. Thus, the separated experiments validate the required key state and media ciphertext interfaces within the measured 4 s window.

4.3.4. Real-World Impact Scenarios. The impact of the attack is a receiver-dependent split view of authenticated media. A malicious participant can send a single broadcast ciphertext that is accepted as cryptographically valid by all receivers, yet causes different receivers to render different audio or video. Thus, the attack does not merely cause packet loss or denial of service; it violates the cross-receiver consistency that secure group communication implicitly relies on, and that has been explicitly studied as a security goal in the group messaging setting [11]. Group key agreement may keep the group state consistent, but this consistency does not automatically extend to the media-encryption layer. Our attack exploits precisely this gap.

The substitution can be confined to a brief but critical segment. In the dual-output (H.265) construction, the malicious sender prepares two valid streams that differ only in their first Y s, where $Y < X$, and are identical thereafter. This choice adds no extra cryptographic operation to the per-frame construction of Section 4.2.2. During the X-second attack window, for example with $X \leq 4$ s, U_{tgt} renders the manipulated content while the other receivers render the benign one. After Y s, the streams converge again, so the visible divergence disappears before the delay window expires.

This enables high-impact equivocation in ordinary call scenarios. In a presentation-style call, the H.265 construction can show two distinct, noise-free videos to different audiences: for example, a slide reading "Buy" to one decision-maker and "Sell" to the rest. The Opus construction can deliver a short adversary-chosen, decision-relevant phrase to U_{tgt}, such as "approve it", "cancel it" or "send it now", while non-target receivers perceive only brief noise that can plausibly be attributed to packet loss within the short delay window.

Because the same ciphertext and tag verify under the corresponding receiver keys, each client locally accepts and renders its media as legitimate. If the victim acts on the manipulated content, the rest of the group observes behavior that appears inexplicable; when confronted, the victim correctly reports what they saw or heard, but no other participant can corroborate it. The attack therefore turns a local authentication success into an evidence-resistant discrepancy among participants, violating the media-level consistency expected in secure group communication.

5. Application to Linphone Group-Call E2EE

This section applies the malicious media playback attack to Linphone [34], an open-source calling application developed by Belledonne Communications SARL. Its messaging and one-to-one calls use standard E2EE mechanisms (Signal through the LIME library[2] and ZRTP with Short Authentication String verification), but group calls use a custom design in which keying material is distributed to participants and media is protected by SRTP-based inner encryption. Since this design is not officially documented, we reconstructed it from the public source code and summarize it as three component protocols (Appendix C, Fig. 11). Unlike the DAVE case study, it shows how the attack changes when media protection follows the packetize-then-encrypt structure of RTP/SRTP.

2. https://gitlab.linphone.org/BC/public/lime

5.1. Inducing Receiver-Dependent Key State

The first requirement is a temporary receiver-dependent key state. Unlike DAVE's MLS epoch transitions, Linphone lets a participant update its own sending media key and redistribute the keying material, which the attacker uses only as a trigger. After U_{mal} updates its sending media key from K_{old} to K_{new}, A_{net} delays the resulting key-update traffic to U_{tgt} while letting it reach the other members. As in DAVE (Section 4), this leaves U_{tgt} on K_{old} and the others on K_{new}, giving U_{mal} the same leverage, a single ciphertext C^* accepted under different receiver keys.

At the implementation level, this receiver-dependent key selection is enabled by the lack of an authenticated, per-packet key identifier. On decryption the receiver simply uses the most recently received master key for a sender, never checking which key a packet actually used. SRTP's optional Master Key Identifier exists for exactly this check, but Linphone leaves it unused.

5.2. SRTP-Constrained Media Protection

The difference from DAVE is the encryption granularity. DAVE encrypts before packetization, so one keystream covers a whole frame, the frame-level unit its H.265 construction exploits (Section 4.2.2). Linphone uses the packetize-then-encrypt order of RTP/SRTP, encrypting each RTP payload independently under a per-packet nonce (Appendix C). Its encryption unit is thus the RTP packet, finer than a frame when a frame is fragmented across packets. This per-packet granularity constrains the constructions that follow. Audio is largely unaffected because an Opus unit fits in one RTP payload, whereas encrypting each video payload as a whole rules out DAVE's two-valid H.265 construction and leaves the same discard-based attack, whose feasibility varies by packetization mode.

5.3. Attacks on Audio and Video

Selective Audio Playback. U_{mal} targets one Opus packet at a time, crafting the SRTP packet so that AES-GCM verification succeeds under both K_{old} and K_{new}. Under K_{new} the same ciphertext decrypts to pseudo-random data, so the Opus payload violates frame-count, size, or padding constraints and is discarded, rendered as silence, or handled through packet-loss concealment. This is the same valid-versus-malformed behavior as the DAVE Opus attack.

Selective Video Playback. Linphone supports video codecs including H.264 and H.265, transported over RTP using payload formats that include Single NAL Unit packets, Aggregation Packets, and Fragmentation Units. Because SRTP encrypts the entire RTP payload, the video attack is structurally identical to the Opus attack rather than to DAVE's H.265 two-valid-stream construction (Section 4.2.2). Unintended-key decryption randomizes the whole payload, including NAL headers, aggregation size prefixes, and FU headers, so no second valid video can survive under K_{new}. The construction is therefore the same valid-versus-discard attack used for Opus in Section 4.2.1. A packet decrypts to a valid H.264/H.265 payload under K_{old} and to a malformed payload that is dropped during depacketization or decoding under K_{new}.

TABLE 3. LINPHONE H.264/H.265 PACKETIZATION UNDER THE PER-PACKET VALID-VERSUS-DISCARD CONSTRUCTION.

Payload format	Adj. blocks/NAL (feasibility)	Tags/frame (computation cost)	Attack
Single NAL Unit	1	≥ 1	$\checkmark_{\text{single}}$
Aggregation Packet	≤ 1	1	$\checkmark_{\text{single}}$
Fragmentation Unit	> 1	> 1	-

In both columns a larger value means a harder attack. Adjustment (Adj.) blocks per NAL > 1 breaks hosting and makes the attack infeasible. Tags per frame > 1 exceeds the one-per-frame floor and raises the generation cost.

Per-Packet Feasibility. Each packet has one keystream and one AES-GCM tag, so the collision fixes exactly one pseudo-random 16-byte adjustment block, which under K_{old} must fall in a decoder-ignored region. Single NAL and Aggregation packets place at most one tag per NAL, so the single block is easily parked in a filler region (Table 3). Fragmentation is the only adverse case. Splitting one NAL across several packets puts one block in each fragment of the reassembled NAL, but only one fits its ignorable tail, so the rest land in bytes the decoder must parse and corrupt even the K_{old} video. Since the malicious participant is the sender, it avoids this by keeping NALs within the MTU or using aggregation. Real-time cost (Table 3) differs only at second order, since generation scales with the video bitrate, not packetization.

5.4. Proof-of-Concept Validation

We implement a fully integrated end-to-end PoC[1] against Linphone group-call E2EE. The PoC establishes an encrypted group call, induces a receiver-dependent key state, and injects ambiguous SRTP packets into the live media stream. In the video attack, the target receiver decrypts and plays the adversary-chosen video under K_{old}, while the other receivers decrypt the same authenticated packets under K_{new} but fail to obtain decodable video. This gives the attack a practical stealth advantage over unauthenticated packet injection. In Linphone, cryptographic verification failures are reflected in the UI connection-quality indicator, whereas video decoding failures after successful authentication are not. Thus, non-target receivers may observe only a brief video stall or frozen frame, without an explicit warning of authentication failure or severe reception degradation.

5.5. Discussion of Other Applications

Beyond Discord and Linphone, Appendix D formulates four enabling conditions for the attack (lack of key commitment, asynchronous key updates, structural tolerance for

ignorable media data, and absence of authenticated key identification) and evaluates further E2EE calling applications against them. The attack applies only when all four hold simultaneously, so failing any one prevents it.

6. Mitigations

The attack becomes possible when receivers temporarily hold multiple candidate media keys, but media packets are not cryptographically bound to the intended key context. We consider two defenses: authenticated key identification as an immediate mitigation, and committing AEAD as a more fundamental one.

Short-term Mitigation: Authenticated Key Identification. The most practical defense is to bind the intended key identifier, epoch, ratchet identifier, or equivalent key-selection metadata to the AEAD input. The sender includes this metadata in the Additional Authenticated Data (AAD), and the receiver uses it to select the corresponding key before verification, rather than trial-decrypting the same ciphertext under multiple candidate keys. Then, a ciphertext authenticated for one key context is rejected under any unintended key context. This principle is already used in modern E2EE systems: MLS [12] authenticates the group epoch, and applications such as Wire [35] and LINE [36] include explicit key identifiers in the AAD of AEAD-protected payloads. The contrast with SFrame [37] is particularly close: when keyed from MLS, SFrame derives its base key from an MLS exporter and builds a separate media framing layer on top of it, as DAVE does, but its frame header carries a key identifier that selects the decryption key and is authenticated as AAD, whereas DAVE carries no corresponding field. Applying the same principle to DAVE's media encryption would remove the missing authenticated key-identification condition exploited by our attack, with negligible overhead. The same requirement applies to systems such as Linphone: key-selection metadata only helps if it is authenticated and validated before decryption is attempted.

Long-term Mitigation: Committing AEAD. A more fundamental defense is to use an AEAD scheme that provides key commitment. This prevents the same ciphertext from being accepted under different keys even if key-identification metadata is incomplete or incorrectly processed. Candidate approaches include generic transforms such as CommitKey [10], KIVR [38], and DNDK [39], as well as natively committing AEAD schemes. For real-time E2EE communication, such schemes must also satisfy low-latency, high-throughput, and low per-packet-overhead requirements.

7. Conclusion

We showed that the lack of key commitment can become practically exploitable in E2EE group real-time communication when combined with asynchronous key updates, tolerant media formats, and absence of authenticated key identification. We demonstrated this attack on Discord's DAVE protocol, including an Opus audio attack and an H.265 video attack, and showed that related audio and video attacks also apply to Linphone.

Our results identify the design conditions under which the threat becomes practical, and show that authenticated key identification or committing AEAD schemes can prevent the attack.

Acknowledgments

This work was supported by JST CRONOS, Japan, Grant Number JPMJCS25N1, JSPS KAKENHI, Grant Number JP24H00696 and JST ACT-X, Japan, Grant Number JPMJAX25M8.

References

[1] J. Salowey, A. Choudhury, and D. McGrew, "AES Galois Counter Mode (GCM) Cipher Suites for TLS," IETF, RFC 5288, 2008, accessed: 2026-06-11. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5288.html

[2] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," IETF, RFC 8446, 2018, accessed: 2026-06-11. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8446.html

[3] C. Perkins, M. Westerlund, and J. Ott, "Media Transport and Use of RTP in WebRTC," IETF, RFC 8834, 2021, accessed: 2026-06-11. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8834.html

[4] E. Rescorla, "WebRTC Security Architecture," IETF, RFC 8827, 2021, accessed: 2026-06-11. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8827.html

[5] D. McGrew and K. Igoe, "AES-GCM Authenticated Encryption in the Secure Real-time Transport Protocol (SRTP)," IETF, RFC 7714, 2015, accessed: 2026-06-11. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7714.html

[6] S. Menda, J. Len, P. Grubbs, and T. Ristenpart, "Context Discovery and Commitment Attacks: How to Break CCM, EAX, SIV, and More," in *Advances in Cryptology – EUROCRYPT 2023*. Springer, 2023, pp. 379–407.

[7] M. Bellare and V. T. Hoang, "Efficient Schemes for Committing Authenticated Encryption," in *Advances in Cryptology – EUROCRYPT 2022*. Springer, 2022, pp. 845–875.

[8] J. Chan and P. Rogaway, "On Committing Authenticated-Encryption," in *Computer Security – ESORICS 2022*. Springer, 2022, pp. 275–294.

[9] Y. Dodis, P. Grubbs, T. Ristenpart, and J. Woodage, "Fast Message Franking: From Invisible Salamanders to Encryptment," in *Advances in Cryptology – CRYPTO 2018*, ser. Lecture Notes in Computer Science, H. Shacham and A. Boldyreva, Eds., vol. 10991. Springer, Heidelberg, August 2018, pp. 155–186.

[10] A. Albertini, T. Duong, S. Gueron, S. Kölbl, A. Luykx, and S. Schmieg, "How to Abuse and Fix Authenticated Encryption Without Key Commitment," in *USENIX Security*, 2022, pp. 3291–3308.

[11] M. Weidner, M. Kleppmann, D. Hugenroth, and A. R. Beresford, "Key Agreement for Decentralized Secure Group Messaging with Strong Security Guarantees," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Association for Computing Machinery, 2021, pp. 2024–2045.

[12] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon, "The Messaging Layer Security (MLS) Protocol," 2023, accessed: 2026-06-11. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9420.html

[13] Discord Inc., "Discord DAVE Protocol Whitepaper," https://daveprotocol.com/, 2024, Protocol Version 1.1, Document Version 1.1.4. Accessed: 2026-06-11.

[14] W3C, "WebRTC Encoded Transform," W3C Working Draft, 2024. [Online]. Available: https://www.w3.org/TR/webrtc-encoded-transform/

[15] F. Dahlgren, J. Doyle, and A. Cygan, "Discord DAVE Protocol Code Review," Trail of Bits, Tech. Rep., 2024.

[16] Belledonne Communications SARL, "Secure Communications using Linphone & Flexisip Solution description," Belledonne Communications SARL, Tech. Rep., 2025. [Online]. Available: https://www.linphone.org/wp-content/uploads/2025/02/Solutions-SecuredCommunications.pdf

[17] Discord Inc., "libdave: Libraries Supporting Discord's Audio & Video End-to-End Encryption (DAVE) Protocol," https://github.com/discord/libdave, 2026, accessed: 2026-06-11.

[18] jagt, "clumsy: a utility for simulating broken network for windows," https://jagt.github.io/clumsy/, 2023, accessed: 2026-06-11.

[19] Google, "Chrome DevTools," https://developer.chrome.com/docs/devtools/, accessed: 2026-06-11.

[20] J.-M. Valin, K. Vos, and T. B. Terriberry, "Definition of the Opus Audio Codec," 2012, accessed: 2026-06-11. [Online]. Available: https://www.rfc-editor.org/info/rfc6716

[21] ITU-T, "Advanced video coding for generic audiovisual services," 2024, accessed: 2026-06-11. [Online]. Available: https://www.itu.int/rec/T-REC-H.264

[22] ——, "High efficiency video coding," 2019, accessed: 2026-06-11. [Online]. Available: https://www.itu.int/rec/T-REC-H.265

[23] FFmpeg Project, "HEVC NAL unit start-code scanning implementation," https://ffmpeg.org/doxygen/trunk/hevc__parser_8c.html, 2026, accessed: 2026-06-11.

[24] GStreamer Project, "GstH265Parser API Reference," https://gstreamer.freedesktop.org/documentation/codecparsers/gsth265parser.html, 2026, accessed: 2026-06-11.

[25] Y.-K. Wang, Y. Sanchez, T. Schierl, S. Wenger, and M. M. Hannuksela, "RTP Payload Format for High Efficiency Video Coding (HEVC)," RFC 7798, 2016. [Online]. Available: https://www.rfc-editor.org/info/rfc7798

[26] D. Dolev and A. C. Yao, "On the Security of Public Key Protocols," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.

[27] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila, "A Formal Security Analysis of the Signal Messaging Protocol," in *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2017, pp. 451–466.

[28] J. Alwen, S. Coretti, Y. Dodis, and Y. Tselekounis, "Modular Design of Secure Group Messaging Protocols and the Security of MLS," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2021, pp. 1463–1483.

[29] R. Canetti, "Universally Composable Security," *Journal of the ACM*, vol. 67, no. 5, pp. 28:1–28:94, 2020.

[30] J. Sherry, S. Hasan, C. Scott, A. Krishnamurthy, S. Ratnasamy, and V. Sekar, "Making Middleboxes Someone Else's Problem: Network Processing as a Cloud Service," in *Proceedings of the ACM SIGCOMM 2012 Conference*, 2012, pp. 13–24.

[31] F. Li, A. A. Niaki, D. Choffnes, P. Gill, and A. Mislove, "A Large-Scale Analysis of Deployed Traffic Differentiation Practices," in *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM)*, 2019, pp. 130–144.

[32] J. Gettys and K. Nichols, "Bufferbloat: Dark Buffers in the Internet," *Communications of the ACM*, vol. 55, no. 1, pp. 57–65, 2012.

[33] E. André, N. Le Breton, A. Lemesle, L. Roux, and A. Gouaillard, "Comparative Study of WebRTC Open Source SFUs for Video Conferencing," in *2018 Principles, Systems and Applications of IP Telecommunications (IPTComm)*. IEEE, 2018, pp. 1–8.

[34] "Linphone - real-time communication," 2026. [Online]. Available: https://www.linphone.org/en/

[35] Wire Germany GmbH, "Wire Security Whitepaper," 2025.

[36] LINE Corporation, "LINE Encryption Overview," 2021, version 2.1.

[37] E. Omara, Justin Uberti, Sergio Garcia Murillo, Richard Barnes, and Youenn Fablet, "Secure Frame (SFrame): Lightweight Authenticated Encryption for Real-Time Media," IETF, RFC 9605, 2024, accessed: 2026-06-11. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9605.html

[38] Y. Naito, Y. Sasaki, and T. Sugawara, "KIVR: Committing Authenticated Encryption Using Redundancy and Application to GCM, CCM, and More," in *Applied Cryptography and Network Security (ACNS 2024)*, 2024.

[39] S. Gueron and T. Ristenpart, "DNDK: Combining Nonce and Key Derivation for Fast and Scalable AEAD," Cryptology ePrint Archive, Paper 2025/785, 2025. [Online]. Available: https://eprint.iacr.org/2025/785

[40] E. Carrara, K. Norrman, D. McGrew, M. Naslund, and M. Baugher, "The Secure Real-time Transport Protocol (SRTP)," IETF, RFC 3711, 2004, accessed: 2026-06-11. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3711.html

[41] Threema, "Threema Cryptography Whitepaper," Threema, Tech. Rep., 2026.

[42] Wire Swiss GmbH, "wire-avs: avs_pc.ts," https://github.com/wireapp/wire-avs/blob/main/wasm/src/avs_pc.ts, 2025, accessed: 2026-06-11.

Appendix A. Derivation of AES-GCM Collision

Here, we describe the mathematical derivation for generating AES-GCM collisions as proposed by Albertini et al. [10]. GHASH evaluates a polynomial with ciphertext blocks as coefficients using a hash subkey r derived from the key K. Assume the adversary knows the two target keys K_1 and K_2 in advance. From these keys, the following components are derived:

- From K_1: Hash subkey r_1 and final offset s_1
- From K_2: Hash subkey r_2 and final offset s_2

For brevity, we omit the AAD and length blocks; including them shifts the exponents uniformly and does not affect the linear structure of the equation. The authentication tag T of AES-GCM is defined for a ciphertext C with m blocks as follows:

$$T = s + \sum_{i=1}^{m} C[i] \cdot r^{m-i+1}$$

The goal of the attack is for a single ciphertext C to output the same tag under two different keys, i.e., to satisfy $T_1 = T_2$. The adversary leaves an arbitrary single block $C[j]$ of the ciphertext as an adjustment variable and fixes all other blocks $C[i]$. In this case, the condition for $T_1 = T_2$ to hold is expressed as the following linear equation:

$$s_1 + C[j] \cdot r_1^{m-j+1} + \sum_{i=1, i\neq j}^{m} C[i] \cdot r_1^{m-i+1}$$

$$= s_2 + C[j] \cdot r_2^{m-j+1} + \sum_{i=1, i\neq j}^{m} C[i] \cdot r_2^{m-i+1}$$

Since GF(2^{128}) is a field of characteristic 2, addition and subtraction are equivalent to XOR, and sign inversion by transposition does not occur. Using this property to solve for $C[j]$ yields the final relation:

$$C[j] \cdot (r_1^{m-j+1} + r_2^{m-j+1})$$
$$= (s_1 + s_2) + \sum_{i=1, i\neq j}^{m} C[i] \cdot (r_1^{m-i+1} + r_2^{m-i+1})$$

The adversary can easily determine the adjustment block $C[j]$ by solving this linear equation, provided the coefficient $r_1^{m-j+1} \oplus r_2^{m-j+1}$ is non-zero. Since $r_1 \neq r_2$, this holds with overwhelming probability for a valid adjustment position j; it can fail only in the negligible case where $r_1^{m-j+1} = r_2^{m-j+1}$ (i.e., r_1/r_2 is a root of unity of order dividing $m - j + 1$).

Appendix B. DAVE's Adversary Model and Security Requirements

B.1. Adversary Model

The whitepaper [13] defines the adversary models for E2EE as Insiders, Outsiders, and Participants. Each is defined as follows:

Definition 2. **(Insiders)** Insiders are individuals such as Discord employees who maintain the cloud infrastructure and have privileged access to Discord's systems.

Definition 3. **(Outsiders)** Outsiders are individuals who do not hold privileged access to Discord's systems, are unauthorized to participate in a specific call, and possess no non-public information regarding that call. They attempt to compromise security by eavesdropping, intercepting, replaying, and modifying network traffic.

Definition 4. **(Participants)** Participants are individuals who are actually participating in the call. This includes past participants who hold a valid invitation to the call, future participants, or individuals who have left and can rejoin.

B.2. Security Requirements

The whitepaper [13] defines the security requirements for E2EE as Confidentiality, Integrity, and Authenticity. Each is defined as follows:

Definition 5. **(Confidentiality)** If only legitimate participants can play the media file, it is guaranteed that information is not leaked to anyone other than the participants authorized to access it.

Definition 6. **(Integrity)** If a media file is received and successfully verified via message authentication, the integrity that the media file has not been modified by unauthorized means is guaranteed.

Definition 7. **(Authenticity)** Participants should be able to clearly identify one another and have sufficient information to trust that they are communicating with the participants they expect.

Appendix C. Linphone Group Call E2EE Protocol

Outer-encryption: In this context, inner-encryption operates within an encrypted communication channel for one-to-one calls. This channel is referred to as outer-encryption, to distinguish it from inner-encryption. However, outer-encryption is not considered in this paper. This is because the encryption for one-to-one communication provided by outer-encryption is not guaranteed to be E2EE in the default configuration. Moreover, even if E2EE is enabled for one-to-one communication, SAS-based authentication is not provided for group calls, and thus man-in-the-middle attacks may be possible against outer-encryption. For these reasons, outer-encryption can be regarded as providing neither confidentiality nor integrity against intermediaries that relay encrypted communication (i.e., Linphone servers and SIP servers relaying media data). Accordingly, we do not consider outer-encryption in this paper.

Protocol for Sharing the EKT Key for Key Distribution

We describe the protocol for sharing the key for key distribution ($mEkt$). $SSPI$ is a random value generated by the server and is used as a key identifier for $mEkt$. $CSPI$ is a random value generated by the representative participant and is later used as the master salt in inner-encryption. The generation of the key MK and the encryption of $mEkt$ using AES-GCM are performed on the representative participant's client device.

$$SSPI \xleftarrow{\$} \{0,1\}^{16}, \tag{1}$$
$$CSPI \xleftarrow{\$} \{0,1\}^{128}, \tag{2}$$
$$mEkt \xleftarrow{\$} \{0,1\}^{256}, \tag{3}$$
$$MK \leftarrow KDF(\tag{4}$$
$$KeyMaterial_{double_ratchet}),$$
$$MK_{key}, MK_{iv} \leftarrow MK[0...32], MK[32...48], \tag{5}$$
$$(C, T) \leftarrow \text{AES-GCM.Enc}(\tag{6}$$
$$MK_{key},$$
$$MK_{iv},$$
$$P = mEkt,$$
$$AAD =$$
$$SSPI \parallel CSPI \parallel DRHeader),$$
$$DRmsg \leftarrow DRHeader \parallel C \parallel T \tag{7}$$

Key Distribution Protocol for Inner-Encryption The sender generates a master key $inner_master_key_{sender}$ for inner-encryption from randomness. The key distribution key $mEkt$ is used as the encryption key and provided as

input to the encryption function AES-KW.Wrap to encrypt $inner_master_key_{sender}$.

The detailed procedure is as follows. Here, SSRC is an identifier for the RTP session, which is randomly generated at RTP session initialization and set by the sender. ROC (Rollover Counter) is a counter variable managed per SSRC. ROC counts the number of rollovers of the RTP sequence number space (i.e., the number of times the 16-bit sequence number space is exhausted) and should not be confused with the RTP sequence number itself.

In the Linphone implementation, SSRC and ROC are encrypted together with $inner_master_key$ using the key distribution key $mEkt$ and distributed to each participant. The final transmitted data is denoted as EKT_{Tag}. It contains the encrypted $inner_master_key_{sender}$, SSRC, and ROC, as well as plaintext metadata, including the key generation $epoch$ of $inner_master_key_{sender}$ (initially 0 and incremented by 1 upon each key update), the length len of EKT_{Tag}, and the $type$ field distinguishing key distribution packets from voice packets with inner-encryption enabled.

$$SSRC \xleftarrow{\$} \{0,1\}^{32}, \tag{8}$$

$$ROC \leftarrow \{0\}^{32}, \tag{9}$$

$$inner_master_key_{sender} \xleftarrow{\$} \{0,1\}^{256}, \tag{10}$$

$$\begin{aligned} Plain \leftarrow\ & Len_{inner_master_key_{sender}} \\ & \| \ inner_master_key_{sender} \\ & \| \ SSRC \\ & \| \ ROC, \end{aligned} \tag{11}$$

$$\begin{aligned} EKTCipher \leftarrow\ & \mathsf{AES\text{-}KW.Wrap}(\\ & K = mEkt, \\ & P = Plain), \end{aligned} \tag{12}$$

$$\begin{aligned} EKT_{Tag} \leftarrow\ & EKTCipher \\ & \| \ SSPI \\ & \| \ epoch \\ & \| \ len \\ & \| \ type \end{aligned} \tag{13}$$

Inner-Encryption Protocol

The SRTP protocol is used as the inner-encryption protocol. In general, SRTP does not guarantee E2EE because SIP servers that relay packets generate and distribute keys. However, in inner-encryption, encryption keys are shared only on users' devices through the above mechanism, and thus the communication can be regarded as E2EE.

The encryption algorithm for media data can be selected from the cipher suites supported by SRTP. In the default configuration, AES-256-GCM is selected. In Linphone, the SRTP implementation is based on `libsrtp`. The referenced version of `libsrtp` depends on the compilation environment of the `liblinphone` library. In this paper, we describe the protocol configuration based on `libsrtp` version 2.7.0, as referenced in BC's OSS repository, and the SRTP configuration in Linphone.

We describe the encryption procedure for media data in inner-encryption. SEQ is a 2-byte RTP sequence number and serves as a counter for the number of packets transmitted within the same RTP stream. For a variable N, (N) denotes the byte length of the variable. $SRTP\text{-}KDF$ is a key derivation function composed of AES in counter mode, as defined in the SRTP specification [40].

$$salt_{SrtpMaster} \leftarrow CSPI[0...12], \tag{14}$$

$$\begin{aligned} inner_key_material \leftarrow\ & inner_master_key_{sender}(32) \\ & \| \ salt_{SrtpMaster}(12) \end{aligned} \tag{15}$$

$$\begin{aligned} K_{enc} \leftarrow\ & \mathsf{SRTP\text{-}KDF}(\\ & M = inner_key_material, \\ & label = 0x00, L = 32 \\ &) \end{aligned} \tag{16}$$

$$\begin{aligned} S_{session} \leftarrow\ & \mathsf{SRTP\text{-}KDF}(\\ & M = inner_key_material, \\ & label = 0x02, L = 12 \\ &) \end{aligned} \tag{17}$$

$$\begin{aligned} in_{nonce} \leftarrow\ & \texttt{0x0000}(2) \ \| \ SSRC(4) \\ & \| \ ROC(4) \ \| \ SEQ(2) \end{aligned} \tag{18}$$

$$nonce \leftarrow in_{nonce} \oplus S_{session} \tag{19}$$

$$\begin{aligned} Header_{RTP} \leftarrow\ & RTP_{version} \\ & \| \ flag_{padding} \\ & \| \ flag_{extension} \\ & \| \ Counter_{CSRC} \\ & \| \ Marker \\ & \| \ PayloadType \\ & \| \ SEQ \\ & \| \ timestamp \\ & \| \ SSRC \end{aligned} \tag{20}$$

$$\begin{aligned} (C,T) \leftarrow\ & \mathsf{AES\text{-}GCM.Enc}(\\ & k = K_{enc}, \\ & P = data_{voip}, \\ & n = nonce, \\ & aad = Header_{RTP}) \end{aligned} \tag{21}$$

$$packet_{data} \leftarrow C \ \| \ T \ \| \ Header_{RTP} \tag{22}$$

Appendix D. Applicability to Other E2EE Applications

The attacks on DAVE and Linphone demonstrate that the lack of key commitment can become practically exploitable in real-time E2EE calling systems. To clarify the boundary of applicability, we survey additional E2EE applications according to four enabling conditions. The attack considered in this paper requires all four conditions to hold simultaneously.

1) **Condition 1 (Lack of Key Commitment).** The media encryption uses an AEAD scheme without key commitment, such as AES-GCM.
2) **Condition 2 (Asynchronous Key Updates).** The key-management layer permits temporary multi-key states during key transitions, for example by retaining old keys for a grace period or allowing receivers to hold multiple candidate keys.
3) **Condition 3 (Structural Tolerance for Ignorable Data).** The media format allows attacker-controlled or corrupted data to be placed in regions ignored by

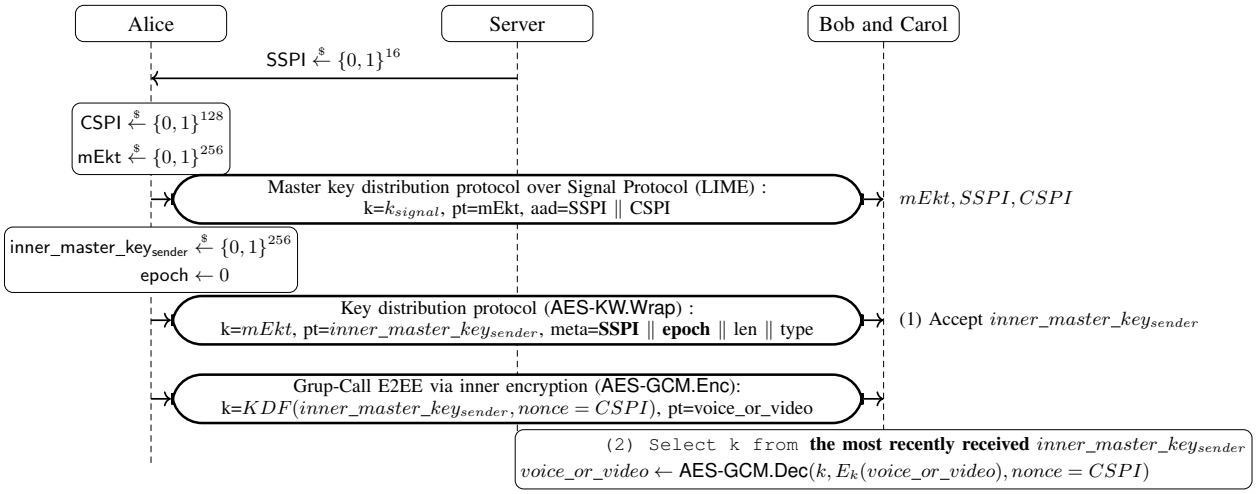


Figure 11. Linphone Group Call E2EE Protocol (Simplified).

TABLE 4. SURVEY OF E2EE APPLICATIONS (INVESTIGATED CONDITIONS 1–4)

Application	Cond. 1	Cond. 2	Cond. 3	Cond. 4
Element (Matrix)	✗	∗	∗	∗
Facebook Messenger	✗	–	–	–
FaceTime	✗	–	–	–
Google Meet	–	–	–	–
KakaoTalk	✗	–	–	–
LINE	✓	∗	∗	✗
iMessage (Apple)	✗	–	–	–
Google Messages	✗	–	–	–
Signal	✗	∗	∗	∗
Silent Phone	–	–	–	–
Skype	–	–	–	–
Telegram	–	–	–	–
Threema	✓	✓	✓	✗
Viber	✗	–	–	–
WhatsApp	✗	–	–	–
Wickr	✓	∗	∗	∗
Wire	✓	✓	✓	✗
Zoom	✓	–	–	–
MS Teams	✓	–	–	–

the decoder, such as Opus padding or H.264/H.265 filler data.

4) **Condition 4 (Absence of Authenticated Key Identification).** The media-encryption layer does not cryptographically bind the intended key identifier, epoch, or equivalent key-selection metadata to the AEAD input. In such designs, a receiver may try multiple candidate keys for the same ciphertext, allowing the same packet to be accepted under different receiver keys.

Table 4 summarizes whether major real-world applications satisfy these four conditions (✓: satisfied, ✗: not satisfied, –: no public information available, ∗: further investigation required).

The purpose of this survey is not to claim that all E2EE calling systems are vulnerable, but to identify when the attack surface exists. Some applications share several design choices with DAVE or Linphone, such as using AES-GCM, supporting Opus, or allowing temporary multi-key states during key transitions. However, satisfying only a subset of the conditions is insufficient. For instance, although several surveyed applications also transmit video using VP8, VP9, or AV1, we examined analogous attacks on these codecs and found that their bitstreams provide far fewer decoder-ignored regions than H.264/H.265 filler data; Condition 3 is therefore not satisfied and the construction does not carry over.

D.1. Wire and Threema

Wire and Threema both satisfy the first three conditions but fail Condition 4, and are therefore not vulnerable.

Wire has recently transitioned its calling infrastructure from Proteus to MLS [35]. Its use of AES-GCM satisfies Condition 1, MLS epoch transitions satisfy Condition 2, and Opus satisfies Condition 3. Threema satisfies Conditions 1 and 3 for the same reasons [41]. Its Condition 2 arises differently: group-call keys are distributed through pairwise one-to-one E2EE channels rather than an MLS group state, which makes it structurally trivial for a malicious initiator to induce inconsistent key states, for example by sending different key-update information to different participants.

Both systems fail Condition 4 because they bind a Key Identifier (KID) to the ciphertext through the AAD. In Wire, the client extracts the unencrypted KID from the header and uses it to look up the corresponding decryption key [42]; the publicly available Threema implementation performs the same check. Because the AAD of an ambiguous ciphertext can carry only one KID, an adversary cannot force different participants to decrypt the same ciphertext under different keys, even if a network adversary delays the key update to strand a target on an old key.

Appendix E.
Ethics Considerations

We followed the principles of responsible disclosure and coordinated vulnerability disclosure throughout this research, and prioritized the safety and privacy of real users when evaluating the feasibility of the attacks. This appendix covers the conduct of our experiments, our disclosure to the affected vendors, and our assessment of the broader impact of publication.

E.1. Experimental Conduct

We did not perform any attack against the live production infrastructure or the real users of any of the services analyzed in this paper. All experiments were conducted in local, controlled environments using accounts and devices owned by the authors, so they did not access other users' data, disrupt the services' infrastructure, or consume third-party resources beyond ordinary client usage. They involved no human subjects and no third-party users, and we did not collect, process, or analyze any personal or user-generated data; accordingly, no institutional review board approval was required. Our analysis was based on publicly available specifications, whitepapers, and open-source code.

For the Linphone experiments, we further isolated the attack-relevant components from production infrastructure. Linphone's SIP server was used only for call-establishment negotiation, while the EKT-based group-key transfer server exercised by the attack was deployed and operated by us. All audio and video traffic, including media packets carrying attack payloads, remained confined to our local network.

E.2. Disclosure

Before submission, we reported our findings, proof-of-concept implementations, and proposed countermeasures to the security teams of all four applications analyzed in this paper.

We reported the Opus audio attack against DAVE to Discord on November 21, 2025, and the H.265 video attack on May 18, 2026. Discord's initial response questioned whether ambiguous ciphertexts could be generated within the protocol's timing constraints; we supplied the measurements of Section 4.2.3 in response. Discord confirmed that the lack of key commitment in AES-GCM was already known to them from the 2024 Trail of Bits audit [15], but that our concrete technique for turning this weakness into a practical attack on DAVE was new to them. On June 13, 2026, they accepted our report as a valid vulnerability, awarded a bug bounty, and stated that they intend to study mitigations based on our report.

We reported the corresponding attacks against Linphone on January 23, 2026 and May 26, 2026. The development team confirmed our analysis and indicated that they plan to incorporate a verified key identifier, for example a combination of key ID and epoch, into the SRTP Master Key Identifier mechanism, which corresponds to the defense of Section 6. We also reported the audio attack to Wire on January 14, 2026 and to Threema on January 28, 2026; the discussions that followed are the basis for their classification in Appendix D.

E.3. Stakeholders and Potential Harm

Following the principles of the Menlo Report, we considered the interests of the affected stakeholders: the end users of E2EE real-time communication services, the service providers and developers who maintain the affected systems, and the wider security research and standardization community. Publishing the attack creates a short-term risk that users could be targeted before fixes are deployed. Coordinating disclosure with each provider in advance, and giving them the opportunity to develop mitigations, serves users' long-term interest in stronger security guarantees while respecting the providers' need to remediate responsibly. We concluded that this benefit, together with alerting the community to the risks of using AEAD without key commitment in real-time communication, outweighs the potential risks of misuse.

We also considered how the techniques and artifacts introduced in this paper could be misused. In principle, the attack could be abused to inject deceptive audio or video into an E2EE group call. It is not, however, a remote or external compromise: it can only be mounted by an adversary who is already an authorized participant in the target call and who can broadcast ciphertext to the group, which bounds both the likelihood and the scope of potential harm. The released artifacts are intended solely for research and evaluation, and demonstrate feasibility under controlled conditions rather than facilitating misuse in production.

Appendix F.
Meta-Review

The following meta-review was prepared by the program committee for the 2027 IEEE Symposium on Security and Privacy (S&P) as part of the review process as detailed in the call for papers.

F.1. Summary

The paper presents an attack on Discord's DAVE protocol used for end-to-end encryption of group audio-video calls. The attack relies on the lack of key commitment in AEAD to construct ciphertexts that decode to different audio-video content for different participants in a call. The authors describe the attack, implement it as a proof-of-concept, and discuss how similar attacks can also apply to other group call systems.

F.2. Scientific Contributions

- Identifies an Impactful Vulnerability
- Provides a Valuable Step Forward in an Established Field

F.3. Reasons for Acceptance

1) The paper identifies an attack on the DAVE protocol, and demonstrates its feasibility by implementing it, although in a controlled environment. The attack causes different participants in a call to receive different audio/video content without noticing it. It targets a widely-deployed system, and has practical impact.
2) The underlying AEAD vulnerability was already known, but was deemed impractical to use, and it was thought that such an attack could not be mounted in real time. The paper shows that that is in fact possible.

F.4. Noteworthy Concerns

1) Because attacks relying on the same technique were already known in an offline context, the novelty lies only in adapting them to work in real-time.
2) The attack was only demonstrated in a controlled lab environment, and not in a real-life setting, though this is justified by ethical considerations.